Consolidator: Learning Persistent Routed Memory Across Context Boundaries

Sungwoo Goo[1] Hwi-yeol Yun[1] Sangkeun Jung[2]
[1]College of Pharmacy, Chungnam National University
[2]Department of Computer Science & Engineering, Chungnam National University
Daejeon, Republic of Korea
swgoo91@gmail.com, hyyun@cnu.ac.kr, hugmanskj@gmail.com

Abstract

Copying short-term memory (STM) into a slower store can preserve state across a context boundary, but persistence alone does not ensure that the retained state influences subsequent memory access. We test this distinction in a Phasor Memory Network (PMNet) using Consolidator, a shared slot-local operator that transforms routed STM before accumulating it into long-term memory (LTM), without replaying the source tokens. After each consolidation, the KV cache and STM are cleared. The retained LTM can still be read and is also fed into the hierarchical router, thereby conditioning which explicit-memory slots subsequent inputs access. We evaluate this mechanism on a two-segment modulo-10 mapping task in which the second segment updates the mapping at the same memory address. Following a second consolidation and reset, a held-out query must recover the updated mapping from LTM. The backbone and memory interface are frozen, leaving only 12.35K Consolidator parameters trainable (0.041% of a 29.95M model). Across five paired runs from the same STM-pretraining checkpoint, direct LTM routing raises updated-mapping recall from $44.38 \pm 1.94\%$ to $87.02 \pm 1.76\%$ ($+42.64 \pm 1.10$ percentage points), while immediate STM recall remains 89.90% in both conditions; both train separate Consolidators and retain the same LTM read paths. Learned consolidation outperforms forced identity accumulation by 21.40 ± 1.91 percentage points without routing and 68.70 ± 1.76 with routing. Thus, on this task, consolidated LTM serves as both retrievable content and an access state that shapes subsequent slot selection.

Keywords: explicit memory, memory consolidation, short-term memory, long-term memory, hierarchical routing, recurrent state

1 Introduction

Transformer context is largely an append-only record of token and KV history [1]. A writable memory offers a different abstraction: experience can be compressed into bounded state, updated at an address, and retrieved after transient context is discarded. Such a system must learn to form useful short-term state, preserve or revise it across boundaries, and use the retained state to select memory slots for later writes and reads.

Merely detaching and copying STM into a slower buffer would provide carryover, but it would leave two mechanistic questions unresolved. Does a fixed pretrained memory interface require a learned transition to revise conflicting content, rather than raw accumulation? Does the retained state only supply values to the read path, or does the router also use it to select slots for subsequent writes and reads?

Differentiable memory, segment recurrence, compression, and adaptive state demonstrate several parts of this lifecycle [2–7]. We ask two linked questions: **can a separately trained operator convert useful routed STM into LTM, and can that LTM guide later slot selection after the KV cache and STM are cleared?**

We study this question in PMNet, where tokens write phase-valued state through a hierarchical router and memory is separable from the local attention cache [8]. Consolidator applies one shared gated phase transform to occupied STM slots, accumulates the result into LTM, and then permits KV and STM to be cleared. In later segments, the model can still read the retained LTM. The router also includes that LTM state in the phase-valued representation used to score slots at the same hierarchy level. As a result, information first written to STM can influence which memory slots later inputs select.

The controlled task comprises two context segments that reuse the same address and rule family, while the second segment replaces the first segment's function parameters. The final held-out query targets this updated mapping and is answered only after both demonstration contexts have been processed and the KV cache and STM have been cleared. In the central intervention, 99.959% of the model is frozen and only the 12.35K-parameter Consolidator is trained. A paired routing ablation retains learned consolidation and the LTM read path but removes the direct LTM input to same-level slot routing. This reduces updated-mapping LTM recall from $87.02 \pm 1.76\%$ to $44.38 \pm 1.94\%$, a paired decrease of 42.64 ± 1.10 pp, while immediate STM recall of the second mapping remains exactly 89.90% in both conditions. The ablation therefore tests whether retained LTM guides later slot selection in addition to supplying retrievable content. A complementary identity intervention tests learned revision against raw STM accumulation: the learned-minus-identity gap is 21.40 ± 1.91 pp without direct routing and 68.70 ± 1.76 pp with routing, while mismatched and fresh LTM controls remain near chance. A dual-objective experiment tests whether pre-consolidation STM recall can coexist with post-reset LTM recall.

Our contributions are:

- A shared slot-local transform that consolidates routed latent STM without replay or topology-dependent parameter growth.
- Direct LTM-conditioned slot routing as an architectural inductive bias that lets retained state guide later writes and reads through a frozen router.
- A sequential same-address update task that separates four memory functions: carrying state across a reset, revising an existing memory, retrieving retained content, and using LTM to guide slot selection. Same-checkpoint, content-replacement, and routing interventions isolate these functions.

The evidence is a mechanism-level proof of concept, not yet a claim about natural-language long context, continual learning, or systems efficiency.

2 Related Work

2.1 Explicit and recurrent memory

Neural Turing Machines and Differentiable Neural Computers established end-to-end learned addressing over external read-write memory [2, 3]. Transformer-XL and Compressive Transformer carry or compress activations across segments [4, 5]; Recurrent Memory Transformer and Infini-attention maintain bounded recurrent state [9, 10]; and Memorizing Transformers retrieve earlier

representations from a non-differentiable store [11]. These systems preserve or retrieve history, whereas our intervention asks a learned slot-local transition to revise conflicting state at the same routed address and then feed the consolidated result directly into subsequent slot routing.

PMNet is the architectural base of this study. It represents recurrent memory updates as phasor rotations and organizes addresses hierarchically [8]. We add an explicit STM–LTM boundary and isolate its function rather than revisiting PMNet's language-modeling or copy benchmarks.

2.2 Adaptive state and compact adaptation

Fast weights, selective state-space models, Test-Time Training layers, and Titans all make computation depend on rapidly changing latent state [6, 7, 12, 13]. Unlike test-time gradient methods, Consolidator keeps model parameters fixed during inference. Its forward pass updates non-parametric memory, which then guides later slot selection. Context Distillation instead stores and routes independent LoRA parameter memories [14]; PMNet writes newly observed content directly into routed non-parametric memory.

Textual Inversion showed that a frozen model can use a small learned embedding to represent a new concept [15]. One-layer post-training likewise found that adapting a restricted Transformer layer can recover much of full-parameter improvement [16]. Both rely on gradient optimization and therefore do not demonstrate forward-only acquisition during a memory episode. They support a more limited capacity premise: pretrained computation can make effective use of a compact adaptation substrate. PMNet trains the memory interface and Consolidator end to end; after training, new content is written and consolidated without changing model parameters.

2.3 Memory consolidation

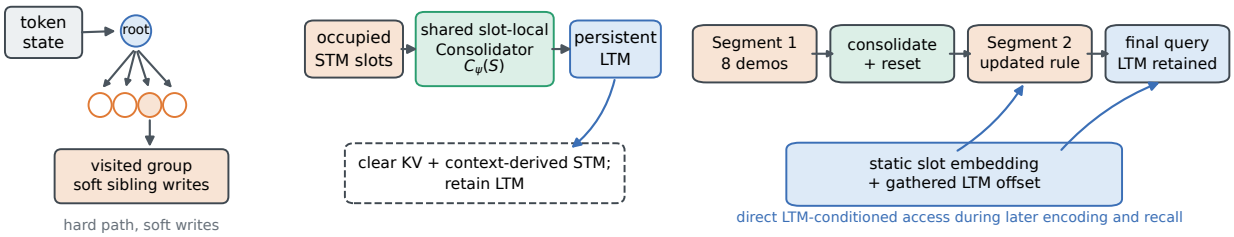
Complementary Learning Systems motivates distinct fast and slow stores, while artificial consolidation commonly combats forgetting through stored or generated replay [17, 18]. Our terminology is an engineering analogy, not a claim of biological equivalence. Auto-Dreamer rewrites symbolic agent memory from stored entries and trajectories [19]. At each boundary, Consolidator transforms the routed STM directly into LTM without revisiting the input sequence. That LTM is later both retrieved as stored content and used to determine which slot new inputs access at the same memory level.

3 Problem Formulation and PMNet Background

3.1 Memory lifecycle

A memory episode comprises segments $X_1, \ldots, X_T$ and three non-parametric states: a local sliding-attention KV cache K_t, routed short-term memory S_t written within segment t, and long-term memory L_t retained across segments. We distinguish two functions of retained LTM: *content state* supplied to the read path and *access state* that conditions which explicit-memory slots subsequent inputs select. Our evaluation asks four questions: whether STM contains the segment-specific mapping before consolidation; whether that mapping can be recovered from LTM after the KV cache and STM are cleared; whether learned consolidation can replace the mapping already stored at a reused address; and whether supplying LTM directly to the router changes post-reset recall relative to making it available only through the read path.

Copying a detached STM snapshot into LTM would alter training credit assignment and preserve state across the reset, but persistence alone would not establish either learned revision or direct



(a) Routed writable STM. (b) Boundary consolidation. (c) Two-segment memory episode and routed reuse.

Figure 1: **Routed latent-memory lifecycle.** Tokens write routed STM; at a boundary, Consolidator updates persistent LTM and KV/STM are cleared. Retained LTM supports later reads and conditions subsequent explicit-memory slot selection.

control over subsequent slot selection. We therefore use two controlled comparisons. Comparing learned consolidation with identity accumulation tests whether a learned transformation at the STM–LTM boundary is needed to revise retained content. Comparing direct LTM routing on and off tests whether consolidated LTM improves recall by guiding subsequent slot selection, beyond supplying stored content through the read path.

3.2 Hierarchical routed writes

PMNet represents memory dimensions as phase angles. At hierarchy block b, a token visits group g with candidate child slots j. Before direct LTM conditioning is introduced, the phase-valued representation used to score each candidate slot is

$$a_{t,b,g,j} = u_{t,b,g,j} + e_{b,g,j}, \tag{1}$$

where u is incoming latent state and e is a learned static slot embedding. With $\phi(a) = [\sin(a); \cos(a)]$, token and slot projections define a cosine-similarity distribution $p_{t,j}$ over siblings. The highest-scoring child selects the next group, while every sibling receives a differentiable phase update

$$\Delta S_{t,j} = p_{t,j}\, \pi \tanh(W_o W_v \,\mathrm{RMSNorm}(h_t))\,. \tag{2}$$

Thus traversal is hard top-1 between levels but writes remain soft within each visited group. An occupancy mask records the groups to consolidate. Reads use the wrapped dynamic state

$$M_{b,g,j} = (S_{b,g,j} + L_{b,g,j}) \mod 2\pi, \tag{3}$$

together with the static embeddings. Further PMNet details are given by Goo et al. [8].

3.3 Operational meaning of replay-free

We call the consolidation procedure replay-free because each demonstration segment is presented only once along the main sequential trajectory that forms and updates LTM. At each boundary, Consolidator receives only routed STM and an occupancy mask; it neither re-encodes the original demonstration tokens nor retrieves them from an episodic replay buffer. After the reset, the final query is answered using retained LTM without presenting the demonstrations again. The dual-objective auxiliary trajectory separately reprocesses each segment's context to measure pre-consolidation STM recall, but it does not alter the replay-free trajectory used to construct LTM.

Replay-free does not mean BPTT-free. The present two-segment experiment retains the differentiable graph across both consolidation boundaries. Detached or truncated training over longer memory episodes remains untested.

4 Learned Latent-State Consolidation

4.1 Shared slot-local phase transform

For an occupied STM slot $S \in \mathbb{R}^{d_m}$, define

$$z(S) = [\cos S; \sin S]. \tag{4}$$

A gated MLP shared across all blocks, groups, and slots produces

$$\begin{aligned} r_\psi(z) &= W_d\,[\mathrm{SiLU}(W_g z) \odot W_u z] + b_d, \\ \begin{bmatrix} c_\psi(S) \\ s_\psi(S) \end{bmatrix} &= z(S) \otimes r_\psi(z(S)), \\ C_\psi(S) &= \mathrm{atan2}(s_\psi(S), c_\psi(S))\,. \end{aligned} \tag{5}$$

where $\otimes$ is element-wise complex multiplication in paired cosine/sine coordinates and atan2 is applied element-wise. The experiments use $d_m = 32$ and hidden dimension $d_c = 64$. Zero output weights and unit-phasor bias make $C_\psi(S) = S$ in Equation (5), so learning begins from exact identity. The transform is slot local and shares 12.35K parameters regardless of tree capacity.

4.2 Persistent accumulation and reset

For hierarchy block b, group g, and child slot j, let $S_{b,g,j}, L_{b,g,j} \in \mathbb{R}^{d_m}$ denote the STM and LTM phase vectors immediately before consolidation, and let $L^+_{b,g,j}$ denote the LTM phase vector afterward. The binary mask $O_{b,g}$ indicates whether group g received an STM write during the current segment. The boundary update is

$$L^+_{b,g,j} = \begin{cases} (L_{b,g,j} + C_\psi(S_{b,g,j})) \mod 2\pi, & O_{b,g} = 1, \\ L_{b,g,j}, & O_{b,g} = 0. \end{cases} \tag{6}$$

If no LTM has previously been stored, $L_{b,g,j}$ is initialized to the zero phase vector. STM, occupancy, and KV state are then cleared while LTM remains. The identity control replaces $C_\psi(S_{b,g,j})$ in Equation (6) with raw $S_{b,g,j}$; its forward update is therefore raw copy-and-accumulate, subject only to phase wrapping. Both modes accumulate rather than overwrite state.

4.3 Direct LTM-conditioned routing

For a subsequent segment, our extension augments the base routing state in Equation (1) with the LTM state of the currently visited group:

$$a_{t,b,g,j} = u_{t,b,g,j} + e_{b,g,j} + L_{b,g,j}. \tag{7}$$

The static embedding remains a parametric address anchor, while LTM becomes an experience-dependent offset. Consolidator receives no route label; task loss trains its output through the existing router and read path. Combining Equations (6) and (7) yields the recurrent path

$$S_t \xrightarrow{C_\psi} L_t \longrightarrow a_{t+1} \longrightarrow S_{t+1}, \tag{8}$$

Table 1: **Procedural rule families.** Function parameters are resampled for each memory episode.

Family	Function	Sampled parameters
ADD10	$y = (x + k) \bmod 10$	$k \in \{1, \ldots, 9\}$
AFFINE10	$y = (ax + b) \bmod 10$	$a \in \{2, \ldots, 9\}$, $b \in \{0, \ldots, 9\}$

so a fixed router can make experience-dependent slot selections because its non-parametric LTM input changes. This path makes LTM an access state rather than only retrievable content.

STM accumulated during a segment is not added directly to the candidate-slot representation used by the router at the same hierarchy level. It can nevertheless influence routing indirectly: reads from earlier hierarchy levels alter the hidden states received by deeper routers. Because LTM remains fixed within a segment, excluding direct same-level STM feedback avoids a token-to-token routing dependency and preserves parallel routing and write aggregation across the segment. Consequently, the routing ablation removes only the direct $L_{b,g,j}$ term in Equation (7): the router itself, LTM retrieval, and indirect influence on deeper levels remain active.

4.4 Objectives

The primary consolidation experiments optimize only the post-reset query for the updated mapping in the second segment:

$$\mathcal{L}_{\text{updated}} = \text{CE}(f(q_2; L_2), y_2). \tag{9}$$

Their checkpoints are selected by validation recall of this updated mapping.

In a separate dual-objective experiment, we test whether one parameter set can support both immediate STM recall and post-reset LTM recall. For each of the two segments, an auxiliary trajectory evaluates its query after the demonstration has formed STM but before consolidation; $\mathcal{L}_{\text{STM}}$ is the mean of these two query losses. Adding this term to Equation (9) gives

$$\mathcal{L}_{\text{dual}} = \mathcal{L}_{\text{updated}} + \mathcal{L}_{\text{STM}}, \tag{10}$$

and selects checkpoints by the mean of updated-mapping LTM recall and pre-consolidation STM recall.

5 Controlled Sequential Same-Address Update Task

5.1 Procedural memory episodes

Each memory episode contains two context segments and one active memory address selected from four address tokens. One rule family, ADD10 or AFFINE10, is sampled per episode; the first and second segments use different function parameters from that family. Parameters are resampled across episodes, preventing a fixed address-to-rule solution.

Each segment provides eight demonstrations and one held-out query, each encoded as

```
[address, rule-family, input x, delimiter, answer y]
```

Loss is applied only to the answer token. Demonstration and query inputs are distinct within each segment, and the final query is selected so that the first and second mappings give different answers. The final prediction therefore cannot receive credit for copying an observed answer or retaining only the stale rule.

Table 2: **Parameter-isolation conditions.** The routing-off condition changes the forward path but has the same trainable parameter count as the standard Consolidator-only condition.

Condition	Consolidation	Trainable subset	Parameters
Learned full	Learned	Entire model	29.95M (100%)
Identity full	Raw STM accumulation	All except Consolidator	29.94M (99.959%)
Consolidator only	Learned	Consolidator only	12.35K (0.041%)
Consolidator only, routing off	Learned	Consolidator only	12.35K (0.041%)
Memory + Consolidator	Learned	Memory read/write/routing + Consolidator	1.526M (5.095%)
Learned full, dual objective	Learned	Entire model	29.95M (100%)

5.2 Two-stage training and evaluation protocol

STM-pretraining stage (Phase 1) trains same-segment rule induction from routed STM. The model processes demonstrations, clears KV history, and answers the held-out query; Consolidator is frozen. All main consolidation-training conditions share the resulting STM-capable checkpoint.

Consolidation-training stage (Phase 2) processes the two demonstration segments sequentially, consolidates STM into LTM after each segment, and then clears the KV cache and STM. The second segment reuses the address with the updated mapping. After the second consolidation and reset, its held-out query is presented without demonstrations or context-derived STM, so only retained LTM can provide the function parameters sampled for that episode. Direct LTM conditioning can affect both slot selection while writing the second-segment update and memory traversal during the final query; the present task measures their combined effect. There is no direct supervision on routes, slots, context tokens, or consolidation boundaries.

5.3 Mismatched-experience intervention

The mismatched control substitutes a donor experience from the same rule family but with different function parameters, then replaces its address token with the recipient's. Format, family, and addressing cue are preserved while memory content changes. Dependence on episode-specific LTM should therefore appear as a collapse in recall.

6 Experimental Setup

6.1 Model and optimization

The 29.95M-parameter model has 12 Transformer layers, hidden dimension 384, and a 128-token sliding-attention window. PMNet uses four hierarchy blocks with branching factor four, 32-dimensional memory, and a 64-dimensional Consolidator hidden layer.

We use AdamW with learning rate 5×10^{-4}, global batch size 256, 100K procedural memory episodes per epoch, and at most 60 epochs. Validation and test each contain 1K memory episodes. Five consolidation-training seeds $\{42, 43, 44, 45, 46\}$ use paired data streams and one fixed held-out test stream. Full architecture, optimizer, software, and seed settings are listed in Appendix E.

6.2 Parameter-isolation conditions

`Learned full` and `identity full` are independently optimized; their difference is not a same-checkpoint causal estimate. The direct mechanism test is `Consolidator only`, which uses direct LTM-conditioned routing: every other parameter is frozen, and the same trained checkpoint is

Table 3: **Same-address update under the Consolidator-only intervention.** Both columns evaluate the same trained checkpoint.

State queried after consolidation	Learned Consolidator	Forced identity
Initial mapping after first consolidation	50.50 ± 3.42	**86.86 ± 0.05**
Updated mapping after second consolidation	**87.02 ± 1.76**	18.32 ± 0.04

evaluated with either learned or forced-identity consolidation. The `Consolidator only, routing off` condition uses the same parameter isolation but removes the direct LTM term in Equation (7); both variants retain learned consolidation, the fixed router, and the LTM read path, and each trains its own Consolidator from the same STM-pretraining initialization.

6.3 Metrics and statistics

Updated-mapping LTM recall is accuracy on the final second-segment query after both demonstration contexts have been consolidated and the KV cache and STM have been cleared. Second-segment immediate STM recall evaluates the updated mapping before the second consolidation; mean immediate STM recall averages the corresponding pre-consolidation queries across both segments. The fresh-LTM control removes persistent memory, whereas the mismatched-LTM control supplies an experience with incorrect function parameters but the correct address and rule family.

We report mean ± sample SD over five seeds and use paired differences for comparisons. Confidence intervals and t-tests are descriptive because $n = 5$. All main consolidation-training seeds share one STM-pretraining checkpoint, so their variation reflects consolidation optimization and data streams conditional on that learned STM representation.

7 Results

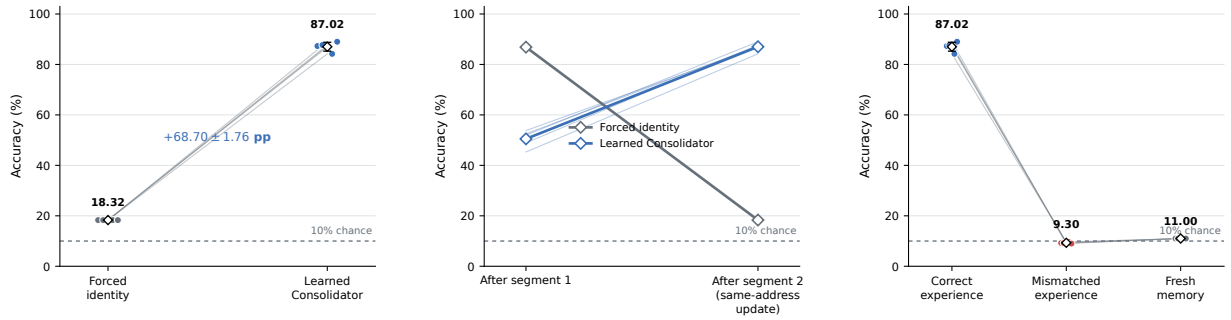
7.1 A learned Consolidator enables persistent updates from frozen STM

The central intervention freezes every STM-pretrained component that forms, routes, and reads STM and trains only the 12.35K-parameter slot transform.

Table 3 and Figure 2 show that identity accumulation transfers the initial mapping but fails after the second same-address write. The learned transform instead reaches $87.02 \pm 1.76\%$ updated-mapping LTM recall, a same-checkpoint gain of 68.70 ± 1.76 pp over forced identity while training 0.041% of the model. Because the backbone, router, and read/write projections are frozen, the gain must pass through the learned boundary transform and existing memory interface. The result also provides independent evidence that the upstream STM is functional: because Consolidator receives only routed STM, its slot-local transform could not recover the rule instantiated in the current memory episode unless that STM already encoded the relevant information. The lower recall of the initial mapping after the first consolidation reflects supervision only on the final updated mapping, not a claim about general retention.

7.2 Consolidated LTM is an access state, not only stored content

Post-reset recall alone does not show that retained LTM affects memory access. We therefore compare two Consolidator-only conditions that retain learned LTM and all read paths but differ in whether the direct LTM term in Equation (7) is present; each condition trains its own Consolidator.



(a) Same-checkpoint boundary intervention.

(b) Identity carries; the learned transform updates.

(c) Recall depends on latent content.

Figure 2: **Consolidator-only intervention.** Paired evaluations compare learned and forced-identity consolidation, the first and second same-address writes, and correct, mismatched, and fresh memory.

Table 4: **Direct LTM-conditioned routing in the Consolidator-only setting.** On and off runs share a byte-identical STM-pretraining initialization and paired consolidation-training seeds.

Direct LTM routing	Learned LTM	Identity LTM	Learned - identity	Segment-2 STM	Mismatched LTM	Fresh LTM
Off	44.38 ± 1.94	22.98 ± 0.04	+21.40 ± 1.91	**89.90 ± 0.00**	9.92 ± 0.69	11.00 ± 0.00
On	**87.02 ± 1.76**	18.32 ± 0.04	**+68.70 ± 1.76**	**89.90 ± 0.00**	9.30 ± 0.24	11.00 ± 0.00

Table 4 and Figure 3 show that direct routing raises updated-mapping LTM recall from $44.38 \pm 1.94\%$ to $87.02 \pm 1.76\%$, a paired gain of 42.64 ± 1.10 pp (95% CI $[41.27, 44.01]$, $p = 1.07 \times 10^{-7}$), while immediate STM recall remains exactly 89.90% in both conditions.

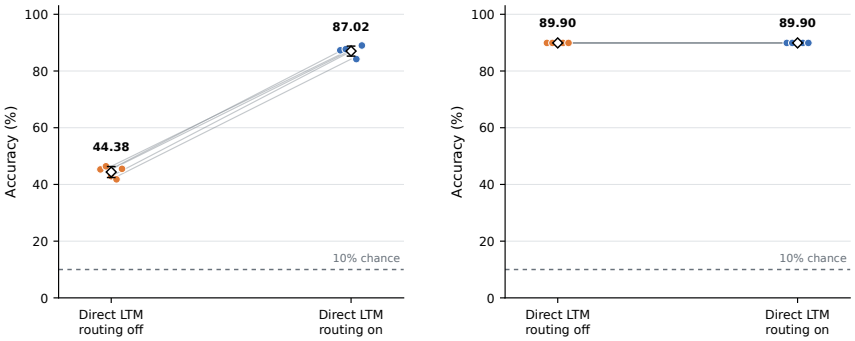
Without direct routing, learned consolidation still exceeds forced identity by 21.40 ± 1.91 pp, showing that LTM remains useful through the read path; direct routing provides the larger additional gain. Thus, consolidated LTM serves as both retrievable content and an access state that guides subsequent slot selection; Appendix A reports per-seed and rule-family results.

7.3 Recall requires the correct experience

Figure 2c and the routing-on row of Table 4 show that updated-mapping LTM recall from the same checkpoint falls from 87.02% with the correct experience to $9.30 \pm 0.24\%$ with a mismatched experience and 11.00% with fresh memory. The mismatch preserves address and rule family, while function parameters vary across memory episodes. Updated-mapping recall therefore depends on the consolidated content rather than a fixed address association or the mere presence of memory.

7.4 Broader parameter-isolation checks

Under the parameter-isolation conditions summarized in Table 2, learned full reaches 93.70% and independently trained identity full reaches 91.34%; their $+2.36 \pm 3.30$ pp difference is not statistically resolved. We therefore do not claim that learned consolidation dominates a fully plastic identity system on this task. Training only the memory path and Consolidator reaches $90.70 \pm 0.51\%$, showing that adaptation can be concentrated in the memory subsystem; Tables C1 and C2 report the aggregate and per-seed results in Appendix C.



(a) Updated-mapping LTM recall. (b) Segment-2 immediate STM recall.

Figure 3: **Direct LTM-conditioned routing.** Lines pair corresponding consolidation-training seeds initialized from the same STM-pretraining checkpoint. Direct routing substantially improves updated-mapping LTM recall (a), while immediate STM recall of the second mapping remains exactly matched (b). The off condition retains learned LTM and all read paths; only direct same-level LTM conditioning of the router is removed.

7.5 Pre-consolidation STM and post-reset LTM recall can coexist

Adding pre-consolidation STM supervision raises mean immediate recall across both segments from $11.39 \pm 0.47\%$ to $95.76 \pm 0.72\%$, while updated-mapping LTM recall reaches $95.58 \pm 0.75\%$ (Figure 4). The two metrics use separate cache trajectories and forward passes, not concurrent queries in one online trajectory. The result therefore shows that one parameter set can support both capabilities under a suitable objective, not that both were jointly read in a single pass; Appendix D reports aggregate statistics and per-seed results.

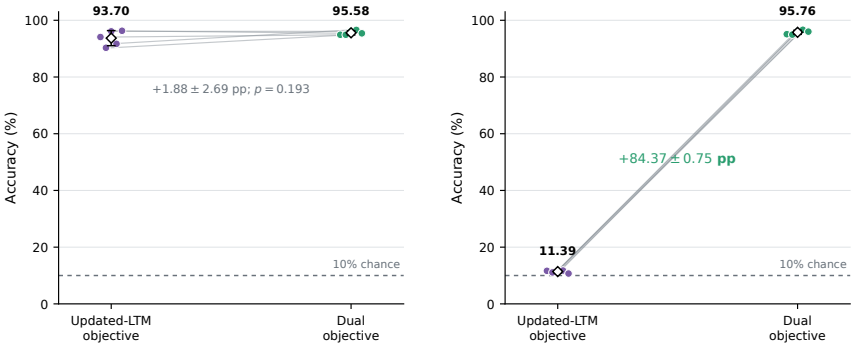
ADD10 is nearly saturated, but the Consolidator-only model also reaches $74.80 \pm 3.07\%$ on AFFINE10; the central result is therefore not explained only by the simpler additive family.

8 Discussion

The results establish a functional chain: STM pretraining forms episode-specific routed state, Consolidator converts it for persistent revision, and replacing the retained experience removes the recall gain. Because the upstream memory interface is frozen and Consolidator receives no source tokens, its success implies that STM already contains the relevant information before consolidation.

The identity and routing interventions show that raw persistence and readout alone do not explain the result: the learned boundary supports conflicting revision, while direct routing provides the larger gain despite matched immediate STM recall. Consolidated LTM therefore functions as both retrievable content and an access state, making its direct connection to the frozen router a task-effective inductive bias for experience-dependent slot selection.

At inference, the fixed Consolidator and router adapt through mutable non-parametric state rather than parameter updates, distinguishing the mechanism from continual fine-tuning and test-time gradient descent.



(a) Updated-mapping LTM recall.

(b) Mean immediate STM recall.

Figure 4: **Pre-consolidation and post-reset recall under a dual objective.** Adding immediate-STM supervision restores mean pre-consolidation recall across both segments (b) without reducing updated-mapping LTM recall (a). The two metrics are evaluated on separate trajectories using one shared parameter set.

9 Limitations

Controlled scope. Memory episodes contain two short context segments, one active address, and modular-arithmetic rules. Both demonstration contexts fit inside the local attention window; the reset isolates persistence but does not test extreme within-segment context, natural language, many competing memories, long horizons, or systems efficiency.

Estimation and provenance. The five main consolidation-training seeds share one selected STM-pretraining representation, so their variance excludes variation from the first training stage. Training retains gradients across the two consolidation boundaries, leaving detached or truncated long-horizon training untested. With $n = 5$, confidence intervals and t-tests are descriptive.

Unisolated design choices. Identity is the principal same-checkpoint control, but we do not compare alternative learned overwrite, EMA, linear, or gated recurrent operators. Boundaries, commit decisions, and eviction are externally specified, and the synthetic task lacks a task-matched external architecture baseline.

Persistence semantics. LTM is initialized for each memory episode; persistence across unrelated sessions, serialization, and deployment restarts is not evaluated. STM, LTM, and consolidation denote computational timescales, not a biological model of memory or sleep.

10 Conclusion

We introduced Consolidator, a shared slot-local transform that converts routed STM into persistent LTM without replaying the source tokens. On a controlled same-address update task, training only its 12.35K parameters while freezing the rest of PMNet yields 87.02% updated-mapping recall, compared with 18.32% when the same checkpoint uses identity accumulation; replacing the retained experience removes this gain. A paired routing ablation further reduces recall from 87.02% to 44.38% while leaving immediate STM recall unchanged, showing that consolidated LTM supports later computation both as retrievable content and as an input to slot selection. These results establish a controlled forward-state adaptation mechanism, not yet a general long-term memory system; detached or truncated long-horizon training, natural language, and scale-up remain open.

Reproducibility

The implementation will be made publicly available at `https://www.github.com/swgoo/pmnet_consolidator`. It records full run configurations, starting-checkpoint hashes, trainable parameter counts, per-seed selected checkpoints, and the fixed test stream shared across conditions. `modeling_pmnet.py` contains the cache, routing, Consolidator, and persistent-state update; `train_pmnet_ablation.py` contains the procedural data generator, STM-pretraining stage (Phase 1), interventions, and consolidation-training runner (Phase 2). Per-seed results and complete hyperparameters are reported in the appendix.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. URL `https://arxiv.org/abs/1706.03762`.

[2] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014. URL `https://arxiv.org/abs/1410.5401`.

[3] Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Alex Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain, Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538:471–476, 2016. doi: 10.1038/nature20101.

[4] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019. URL `https://arxiv.org/abs/1901.02860`.

[5] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. *arXiv preprint arXiv:1911.05507*, 2019. URL `https://arxiv.org/abs/1911.05507`.

[6] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2024. URL `https://arxiv.org/abs/2407.04620`.

[7] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024. URL `https://arxiv.org/abs/2501.00663`.

[8] Sungwoo Goo, Hwi-yeol Yun, and Sangkeun Jung. Phasor memory networks: Stable backpropagation through time for scalable explicit memory. *arXiv preprint arXiv:2605.13370*, 2026. URL `https://arxiv.org/abs/2605.13370`.

[9] Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer. *arXiv preprint arXiv:2207.06881*, 2022. URL `https://arxiv.org/abs/2207.06881`.

[10] Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave no context behind: Efficient infinite context transformers with infini-attention. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL https://arxiv.org/abs/2404.07143.

[11] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022. URL https://arxiv.org/abs/2203.08913.

[12] Jimmy Ba, Geoffrey Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems*, 2016. URL https://arxiv.org/abs/1610.06258.

[13] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023. URL https://arxiv.org/abs/2312.00752.

[14] Zangwei Zheng, Zhen Li, Xuehai Wen, et al. Context distillation as latent memory management. *arXiv preprint arXiv:2605.28889*, 2026. URL https://arxiv.org/abs/2605.28889.

[15] Rinon Gal, Yuval Alaluf, Yuval Atzmon, Or Patashnik, Amit H. Bermano, Gal Chechik, and Daniel Cohen-Or. An image is worth one word: Personalizing text-to-image generation using textual inversion. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=NAQvF08TcyG.

[16] Zijian Zhang, Rizhen Hu, Athanasios Glentis, Dawei Li, Chung-Yiu Yau, Hongzhou Lin, and Mingyi Hong. Is one layer enough? training a single transformer layer can match full-parameter RL training. *arXiv preprint arXiv:2607.01232*, 2026. URL https://arxiv.org/abs/2607.01232.

[17] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3): 419–457, 1995. doi: 10.1037/0033-295X.102.3.419.

[18] Tyler L. Hayes, Giri P. Krishnan, Maxim Bazhenov, Hava T. Siegelmann, Terrence J. Sejnowski, and Christopher Kanan. Replay in deep learning: Current approaches and missing biological elements. *arXiv preprint arXiv:2104.04132*, 2021. URL https://arxiv.org/abs/2104.04132.

[19] Chenchen Ye, Yejin Liu, Yujie Wang, et al. Auto-dreamer: Learning offline memory consolidation for language agents. *arXiv preprint arXiv:2605.20616*, 2026. URL https://arxiv.org/abs/2605.20616.

A Direct LTM Routing by Seed and Rule Family

Table A1 reports the pooled paired routing results for each seed. Table A2 then stratifies learned-LTM recall by rule family. Direct LTM routing improves both families: the simpler ADD10 family approaches saturation, while AFFINE10 retains a paired gain of 38.56 ± 1.29 pp. The overall routing effect is therefore not attributable only to the additive rules.

Table A1: **Per-seed direct LTM-routing ablation in the Consolidator-only setting.** Routing-on and routing-off runs start from the byte-identical Phase-1 checkpoint and use paired Phase-2 seeds, but each trains its own Consolidator. *Off learned LTM* is updated-mapping recall after both consolidations and the final reset with direct same-level LTM routing disabled; *Off identity LTM* reevaluates that same routing-off checkpoint using forced identity accumulation, and *Learned–identity* is their same-checkpoint difference. Segment-2 STM, mismatched LTM, and fresh LTM are also measured in the routing-off condition. *On–Off* pairs learned-LTM recall from the standard routing-on run with learned-LTM recall from the routing-off run. Both routing conditions retain the fixed router and all LTM read paths. All values are percentages on the shared fixed test stream.

Seed	Off learned LTM	Off identity LTM	Learned - identity	Segment-2 STM	Mismatched LTM	Fresh LTM	On - Off
42	45.30	23.00	+22.30	89.90	9.30	11.00	+42.00
43	46.40	23.00	+23.40	89.90	10.40	11.00	+41.30
44	42.90	23.00	+19.90	89.90	10.70	11.00	+44.00
45	41.80	22.90	+18.90	89.90	9.10	11.00	+42.40
46	45.50	23.00	+22.50	89.90	10.10	11.00	+43.50
Mean	44.38	22.98	+21.40	89.90	9.92	11.00	+42.64
SD	1.94	0.04	1.91	0.00	0.69	0.00	1.10

Table A2: **Direct LTM-routing ablation by rule family.** Routing-off and routing-on values are updated-mapping LTM recall from separately trained Consolidator-only runs that share the byte-identical Phase-1 checkpoint and paired Phase-2 seeds. On–Off differences are computed within each seed before aggregation. Values are percentages, reported as mean ± sample SD over five seeds on the shared fixed test stream.

Rule family	Routing off	Routing on	Paired On–Off
ADD10	52.52 ± 1.37	**99.01 ± 0.60**	**+46.49 ± 1.23**
AFFINE10	36.24 ± 2.61	**74.80 ± 3.07**	**+38.56 ± 1.29**

B Exact Memory-Episode Procedure

B.1 Pseudocode

```
sample family f in {ADD10, AFFINE10}
sample address a from four split tokens
sample first-segment parameters theta_1
sample second-segment parameters theta_2 != theta_1

for segment d in {1, 2}:
    sample 8 distinct demonstration inputs
    sample 1 held-out query input
    if d == 2:
        require f_theta_1(query) != f_theta_2(query)
    form 40-token demonstration context
    process context with routed STM writes
    consolidate occupied STM groups into LTM
    clear KV and STM; retain LTM

present the second-segment held-out query without demonstrations
apply loss only to the answer token
```

For the dual objective, a separate immediate-STM trajectory reprocesses each segment's context and adds its pre-consolidation query loss to the updated-mapping LTM loss.

C Per-Seed Core Results

Table C1: **Core parameter-isolation ablations.** Percent accuracy, mean ± SD over five seeds; LTM recall follows the final reset.

Condition	Updated LTM	Fresh	Mismatched	Segment-2 STM
Learned full	**93.70 ± 2.66**	10.84 ± 0.54	7.70 ± 0.98	11.94 ± 0.67
Identity full	**91.34 ± 2.90**	11.12 ± 0.28	7.92 ± 1.02	18.16 ± 4.34
Consolidator only	**87.02 ± 1.76**	11.00 ± 0.00	9.30 ± 0.24	**89.90 ± 0.00**
Memory + Consolidator	**90.70 ± 0.51**	10.30 ± 0.29	9.44 ± 0.31	14.88 ± 3.24

Table C2: **Fixed-test-stream results for all 20 core runs.** "Alternative mode" is a same-checkpoint diagnostic that changes only the consolidation operator at evaluation: learned conditions use forced raw-identity accumulation, whereas identity full uses its frozen, identity-initialized Consolidator path. The alternative mode is not independently trained.

Condition	Seed	Primary recall	Alternative mode	Fresh	Segment-2 STM	Mismatched
Learned full	42	94.10	53.20	10.90	11.50	7.60
Learned full	43	90.30	87.90	10.80	11.50	9.30
Learned full	44	96.10	82.90	11.00	12.20	6.90
Learned full	45	91.70	88.50	11.50	13.00	7.80
Learned full	46	96.30	75.10	10.00	11.50	6.90
Identity full	42	96.10	96.10	11.40	12.60	7.10
Identity full	43	88.20	88.10	11.10	20.20	9.10
Identity full	44	90.70	90.70	10.80	23.70	8.70
Identity full	45	91.20	91.10	10.90	19.10	6.70
Identity full	46	90.50	90.20	11.40	15.20	8.00
Consolidator only	42	87.30	18.30	11.00	89.90	9.20
Consolidator only	43	87.70	18.30	11.00	89.90	9.20
Consolidator only	44	86.90	18.40	11.00	89.90	9.50
Consolidator only	45	84.20	18.30	11.00	89.90	9.60
Consolidator only	46	89.00	18.30	11.00	89.90	9.00
Memory + Consolidator	42	91.60	85.00	10.50	10.70	9.20
Memory + Consolidator	43	90.60	87.60	10.10	16.70	9.10
Memory + Consolidator	44	90.40	72.70	10.40	18.90	9.40
Memory + Consolidator	45	90.40	89.30	9.90	12.70	9.70
Memory + Consolidator	46	90.50	82.10	10.60	15.40	9.80

D Per-Seed Dual-Objective Results

This experiment asks whether the low immediate-STM recall observed when training only for the final post-reset query reflects an architectural incompatibility between STM and LTM, or simply the absence of direct STM supervision. Both conditions use the fully trainable model, share the same Phase-1 STM-pretraining checkpoint, and pair the Phase-2 seeds and data streams. They differ in their training objective and checkpoint-selection metric, so this comparison is between independently optimized objectives rather than a same-checkpoint intervention.

The *Updated LTM only* condition optimizes the final second-segment query after both consolidations and the final KV/STM reset. The *Updated LTM + STM* condition adds the mean loss of two auxiliary pre-consolidation queries, one for each segment. These immediate-STM queries are evaluated on separate auxiliary trajectories; they share model parameters with the persistent-memory trajectory but do not interrupt or supply information to it. In the tables, *Updated LTM* is final updated-mapping recall, *Mean STM* averages immediate recall across the two segments, and *Segment-2 STM* reports immediate recall of the updated mapping alone.

Table D1: **Aggregate dual-objective experiment.** Paired runs share seeds and STM-pretraining provenance; the consolidation-training objective and validation monitor differ.

Training objective	Updated LTM	Mean STM	Segment-2 STM	Mismatched	Fresh
Updated LTM only	93.70 ± 2.66	11.39 ± 0.47	11.94 ± 0.67	7.70 ± 0.98	10.84 ± 0.54
Updated LTM + STM	**95.58 ± 0.75**	**95.76 ± 0.72**	**95.86 ± 0.91**	8.54 ± 0.30	10.96 ± 0.67
Paired change	+1.88 ± 2.69	**+84.37 ± 0.75**	**+83.92 ± 0.59**	—	—
95% CI	[-1.46, +5.22]	**[+83.44, +85.30]**	**[+83.19, +84.65]**	—	—
Paired p	0.193	1.52×10^{-9}	5.83×10^{-10}	—	—

Adding direct STM supervision raises mean immediate recall by 84.37 ± 0.75 pp while retaining $95.58 \pm 0.75\%$ updated-mapping LTM recall. The $+1.88 \pm 2.69$ pp change in updated LTM recall is not statistically resolved; the result therefore supports coexistence of the two capabilities under one parameter set, not an improvement in LTM attributable to the auxiliary objective. The paired confidence intervals and t-tests are descriptive because $n = 5$.

Table D2: **Per-seed dual-objective results.**

Seed	Best epoch	Validation dual	Test dual	Updated LTM	Mean STM	Forced identity
42	10	95.07	95.00	94.90	95.10	84.60
43	14	95.42	94.92	94.90	94.95	72.30
44	20	95.90	96.10	96.10	96.10	89.20
45	22	96.20	96.63	96.60	96.65	67.30
46	16	95.10	95.70	95.40	96.00	71.70

For the per-seed table, *Validation dual* and *Test dual* are the arithmetic means of updated-mapping LTM recall and mean immediate-STM recall on their respective splits. *Forced identity* reevaluates the selected dual-objective checkpoint with raw identity accumulation in place of the learned Consolidator; it is a same-checkpoint diagnostic, not a separately trained identity condition.

E Hyperparameters and Architecture

Table E1: **Model, optimization, and hardware settings.**

Category	Setting
Backbone initialization	PMNet copy-task checkpoint, followed by same-segment STM rule induction (Phase 1)
Total parameters	29.95M
Transformer	12 layers; hidden size 384; FFN size 1,024; 12 query and 12 KV heads
Local attention	Sliding window 128; attention dropout 0.1; RMSNorm $\epsilon = 10^{-6}$
Memory hierarchy	Four blocks; branch factor 4; 85 routing groups; 340 candidate slot vectors
Memory features	Phase dimension 32; four read heads; writes at layers 0, 3, 6, and 9
Consolidator	Intermediate size 64; SiLU gate; 12.35K shared parameters; identity phase initialization
Optimizer	AdamW; $\beta = (0.9, 0.95)$; learning rate 5×10^{-4}; weight decay 0.1
Schedule	100 warmup steps, then cosine decay
Gradient clipping	Global norm 1.0
Precision and batch	`bf16-mixed`; global batch size 256; one device
Hardware	One NVIDIA RTX 4090; CUDA 13.0; FlashAttention 2
Software	Python 3.12.3; PyTorch 2.10.0+cu130; Transformers 5.14.1; Lightning 2.6.5
Training budget	100K memory episodes/epoch; at most 60 epochs; patience 6
Evaluation	1K validation and 1K fixed test memory episodes
Phase-2 seeds (consolidation)	42, 43, 44, 45, 46
Temporal graph	`detach_long_term_between_sleeps=False` for reported adaptation runs